



Incus & incus-compose

Modern Linux Container & VM Orchestration Without Docker

Speaker: René Jochum

Linux Containers (**lxc**)



The Landscape Today: Two Separated Worlds

For over a decade, Linux virtualization and containerization has been split:

- **Application Containers (OCI / Docker / Podman):**
 - Great developer UX, `compose.yaml` standard
 - Ephemeral microservices, layered image registries
- **System Containers & Virtual Machines (LXC / Incus / KVM):**
 - Full OS environments, systemd init, persistent servers
 - Native ZFS/Btrfs/Linstor storage, real networking, high security

The developer reality: The world adopted Compose for packaging stacks.



The Pain Points We Just "Accepted"

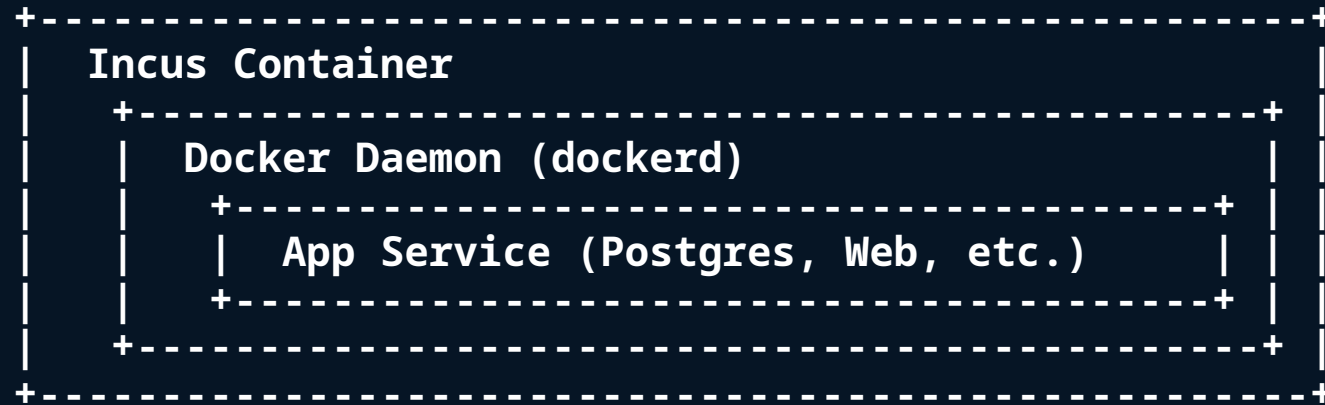
When running Docker in production or development:

- **The Root Daemon:** A monolithic daemon running with root privileges
- **Network Masquerading:** Tangled `iptables` / `nftables` rules that bypass UFW and complicate routing
- **Desktop Overhead:** On macOS and Windows, Docker Desktop requires a heavy, battery-draining virtual machine (or WSL2) just to run a daemon
- **Storage Inefficiencies:** Overlayfs piled on top of host filesystems with fragile snapshotting and permissions headaches



The Common Workaround: Docker inside LXC

People who love LXC / Incus often do this:



- **Two runtimes** doing one job
- **Nested namespaces** add subtle failure modes
- Often requires **privileged** containers or security concessions for nested overlays
- Filesystem on top of filesystem wastes I/O and disk space



Meet Incus: The Unified Linux Infrastructure Engine

- **Community-driven fork of LXD** under the **Linux Containers** (linuxcontainers.org) project
- **Apache 2.0 license**, fully open development
- **One single daemon** (`incusd`), one clean REST API
- **Runs three workloads natively side-by-side:**
 - OCI Application Containers** (microservices, no nested daemon)
 - LXC System Containers** (full OS with systemd/OpenRC)
 - Hardware Virtual Machines** (KVM/QEMU)



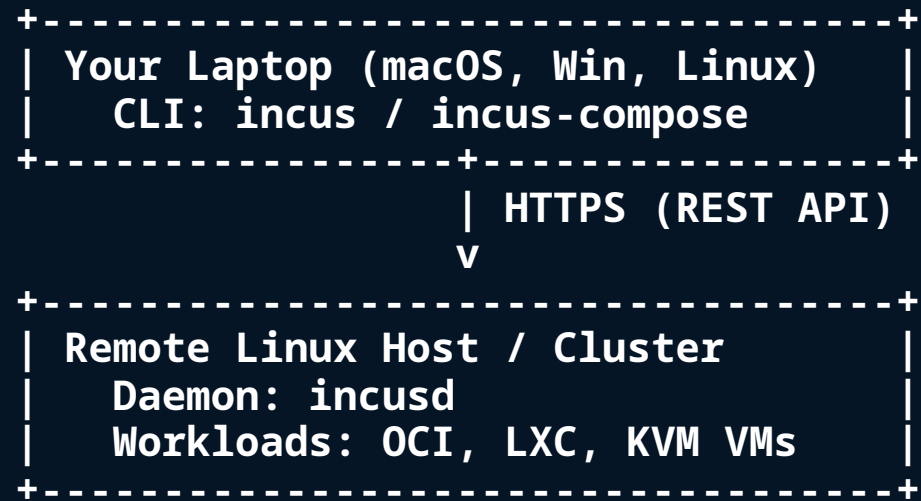
Why Incus Excels

Capability	Classic OCI Engines (Docker)	Incus
	Namespaces + cgroups; daemon is root	, AppArmor + seccomp
Workloads	Application containers only	OCI apps, system containers, and KVM VMs
	Layered overlayfs	
Networking	iptables port mapping	Real bridge/OVN IPs, proxy devices, direct routing
	Flat namespace	, quotas, resource limits



True Client-Server Architecture

Incus was designed from day one as a network-first API:



- The client is a **single cross-platform Go binary**
- Talk to local or remote Incus servers directly over **TLS / HTTPS**
- **Zero local VMs needed on macOS or Windows**



The Missing Link

Incus has native OCI registry support:

- It can pull images directly from `docker.io`, `ghcr.io`, or any registry
- It can run an OCI container natively: unprivileged, with no nested Docker daemon

So why isn't everyone using it?

- Incus is an **infrastructure manager** (`incus launch`, `incus network create`)
- It does not understand multi-service stacks, dependencies, or `.env` files
- Writing 15 imperative `incus` commands to bring up a database, redis, and web app is tedious

→ **We needed Compose semantics on Incus.**



Enter **incus-compose**

A drop-in replacement for **docker compose** powered by Incus.

```
incus-compose up -d
```

- **Standard Compose Spec:** Parsed using the official **compose-go** library (the same parser Docker Compose uses)
- **Zero rewrites:** Point it at your existing **compose.yaml**
- **Native Incus:** Translates services into unprivileged Incus OCI instances, managed networks, and native storage volumes
- Part of the official **Linux Containers (lxc)** organization



Drop-in Compatibility

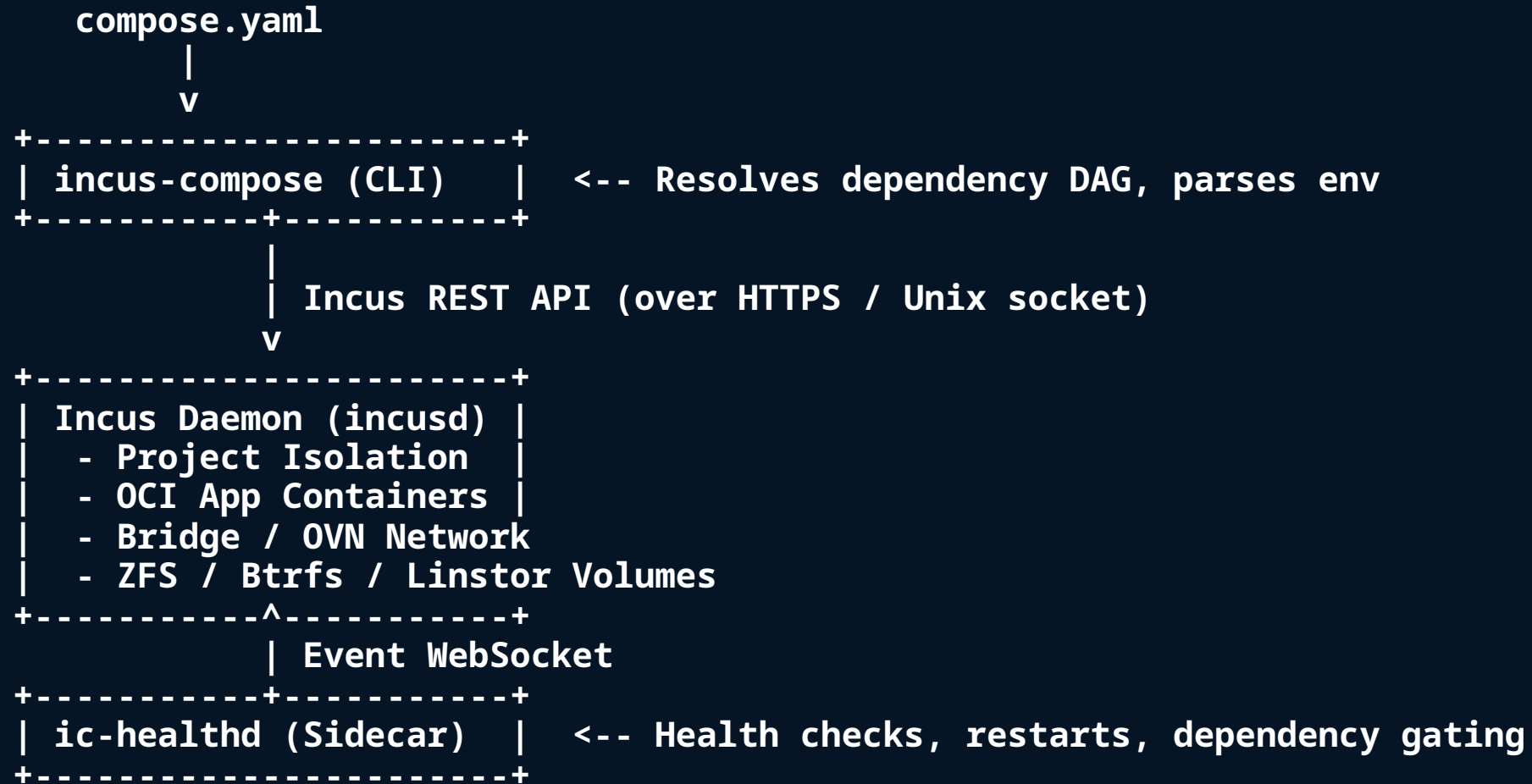
All the standard commands you already know:

```
incus-compose up -d           # Start stack in background
incus-compose ps              # List project instances & status
incus-compose logs -f [service] # Follow aggregated or service logs
incus-compose exec web sh     # Shell into a running instance
incus-compose top             # Inspect running processes
incus-compose down -v         # Stop and clean up (volumes optional)
```

Plus: `start`, `stop`, `restart`, `pause`, `unpause`, `run`, `cp`, `build`, `config`.



How It Works Under the Hood





Solving the Hard Problem: `ic-healthd`

Incus has no native healthcheck or restart policy engine.

How does `depends_on: { condition: service_healthy }` work?

- `ic-healthd`: A lightweight sidecar daemon running on the Incus host
- **Single Event Stream**: Connects to Incus via WebSocket events (`/1.0/events`)
- **Zero Polling Overhead**:
 - Detects when instances start, stop, or crash
 - Runs defined `test:` commands inside the containers
 - Updates `user.healthcheck.status` metadata
 - Automatically restarts failed services per restart policy
 - Signals `incus-compose` to unblock dependent services



Storage: Real Volumes & Seeding

- **Native Storage Engine Backing:**

- In incus-compose, named volumes are **real Incus custom storage volumes** on ZFS, Btrfs, Ceph, LVM, or Linstor (DRBD)

- **Image Seeding (Just Like Docker):**

- When an image declares a `VOLUME` or specifies a mount like `conf:/etc/nginx/conf.d`:
- `incus-compose` extracts the initial image data into the new volume on first run
- No more empty configs breaking your services

- **UID/GID Shifting:**

- Clean translation between host and unprivileged container IDs



Superpower 1: Native Incus Extensions (`x-incus`)

Need raw Incus features that Compose spec doesn't have? Use escape hatches:

```
services:
  ai-worker:
    image: nvidia/cuda:12.4.0-base-ubuntu22.04
    x-incus:
      devices:
        gpu:
          type: gpu
    x-incus-compose:
      limits:
        cpu: "8"
        memory: 16GiB
```

- Direct **GPU passthrough** (NVIDIA, AMD, Intel)
- **Project-wide resource limits** enforced by the daemon
- Custom device passthrough (USB devices, raw disk blocks, network interfaces)



Right Tool for the Job: `incus-apply` vs `incus-compose`

Should you put system containers or VMs into `compose.yaml`? No.

- **Compose is for Application Stacks (`incus-compose`):**
 - OCI microservices, application ports, volume mappings, dependencies
 - Forcing system containers (systemd init) or KVM VMs into Compose stretches the spec past its design
- **Infrastructure as Code belongs in `incus-apply` :**
 - Declarative management for **system containers, KVM VMs, profiles, networks, and storage pools**
 - Keep infrastructure definition cleanly separated from application stacks



Superpower 2: Building OCI Images Made Effortless

Building local container images on Incus without `incus-compose` is **painful**:

- **Incus is a hypervisor, not a builder:** No native `Dockerfile` parser
- **The Manual Chore:** Build locally → export image to a tarball → run `incus image import` → tag it → recreate the container manually. Every single code edit!

With `incus-compose build`:

```
services:
  web:
    build: .
    # or: build: { context: ., dockerfile: Containerfile }
```

```
incus-compose up --build
```

- **Seamless pipeline:** Detects `buildah`, `podman`, or `docker` on your machine
- **Direct Incus Import:** Builds the rootfs, streams it into the Incus project over the REST API, and restarts



Superpower 3: Stack-Aware Backups

```
# Create an atomic backup of all project volumes  
incus-compose backup create
```

```
# List available restore points  
incus-compose backup list
```

```
# Restore stack state from a backup snapshot  
incus-compose backup restore 20260923-010000
```

- Snapshots all named volumes and copies them into `<project>-backup`
- Backups live in an **isolated Incus project**: safe from `down -v` accidents!
- Includes `verify` and automated pruning (`--keep-last N`)



Superpower 4: Private Port Forwarding

Ever had a database or internal admin panel that you didn't want to expose to the world?

```
incus-compose port-forward db 5432
```

- Listens on `localhost:5432` on your workstation
- Tunnels TCP traffic securely through the **Incus HTTPS API** into the instance
- The container port **never needs to be published** in `ports:` or exposed on the host!
- Works from your Mac or Windows laptop to remote servers



Air-Gapped and Fast Image Operations

- **Two-stage Image Cache:**
 - Pulls once, caches locally
 - Survives `down` and `up` cycles
 - Dodges Docker Hub / registry rate limits
- **True Air-Gapped Operation:**
 - `pull` is the *only* command that reaches out to the registry
 - `incus-compose up --pull never` guarantees zero outbound calls
 - Perfect for restricted enterprise networks, air-gapped environments, and CI



Real-World Example: Immich (`examples/immich/`)

A complex, 5-service production stack:

```
services:
  server:
    image: ghcr.io/immich-app/immich-server:${IMMICH_VERSION:-release}
    ports: ["2283:2283"]
    depends_on:
      redis: { condition: service_healthy }
      database: { condition: service_healthy, restart: true }
      machine-learning: { condition: service_healthy }
      microservices: { condition: service_healthy }

  machine-learning:
    image: ghcr.io/immich-app/immich-machine-learning:${IMMICH_VERSION:-release}
    volumes: [model-cache:/cache]

  database:
    image: ghcr.io/immich-app/postgres:14-vectorchord...
    volumes: ["${DB_DATA_LOCATION}:/var/lib/postgresql/data"]
```



Tuning with `compose.incus.yaml`

Keep upstream `compose.yaml` clean, customize with Incus overrides:

```
# compose.incus.yaml (overrides examples/immich/compose.yaml)
services:
  server:
    networks:
      default:
        ipv4_address: ${SERVER_IPV4}/${IPV4_NETMASK}

volumes:
  library:
    x-incus-compose:
      pool: ${UPLOAD_POOL} # Place massive photo library on bulk ZFS pool
```

- **No editing upstream compose files:** overrides merge cleanly
- **Storage pool steering:** Fast NVMe for PostgreSQL, bulk ZFS HDD pool for photos
- **Fixed IPs:** Direct static IP assignments on managed Incus bridges



Live Demo: Immich in Action

1. Launch the stack:

```
incus-compose up -d
```

(5 containers spin up in parallel; `ic-healthd` gates the API server until Postgres, Redis, and ML report healthy)

2. Inspect & Verify:

```
incus-compose ps      # Shows compose health status
incus list             # Shows unprivileged Incus containers with real IPs!
```

3. Instant Volume Backup:

```
incus-compose backup create  # ZFS snapshot of library + DB volumes
incus-compose backup list    # Stored safely in immich-backup project
```



Where Things Go: The Expanding Ecosystem

The Incus ecosystem is growing beyond the CLI:

- **Dynamic Ingress:** `incus-caddy-config`
 - Zero-touch reverse proxy for Caddy on Incus
 - Subscribes to Incus lifecycle events via `ievent`
 - Stages Caddyfiles via SFTP directly to the storage volume (`/config`), formats with `caddy fmt`, atomic swap, and `caddy reload`
 - Auto-discovers and load-balances instances via labels:
`edge: "photos.example.com,upstream=2283"`
- **Split-Horizon DNS:** `ic-dns` (in `develop`)
 - Integrated authoritative DNS server (`incus-compose dns {up|down|status|logs}`)
 - Resolves service names across compose projects and hosts cleanly via `x-incus-compose.dns`



Where Things Go: Clustering & An Operator?

- **True Clustering Support (Predictable Workloads):**
 - Native Incus cluster integration
 - **Core principle:** Containers stay **pinned to their assigned node** for data locality, storage stability, and operational sanity
 - **Never migrate or shuffle containers around** during normal operation—failover only occurs upon host crash
- **ic-healthd as the Operator?:**
 - Stacks only *declare* desired state and graph edges on instance metadata (`user.operator.*`)
 - CLI becomes an **applier + watcher**; kill CLI mid-`up` and the stack still converges
 - Daemon acts as **reconciler**: enforces topological boot ordering after host reboots (`boot.autostart: false`), cascading restarts (`restart: true`), and drift correction



When to Use What: An Honest Assessment

Stick with Docker / Podman when:

- You are deploying to managed Kubernetes clusters (EKS, GKE, K3s)
- You depend on cloud provider proprietary container engines (AWS ECS, Google Cloud Run)
- You rely on Docker Desktop GUI extensions on developer machines

Choose Incus + incus-compose when:

- You run Linux servers: VPS, bare metal, homelabs, edge nodes
- You care about **security by default** (unprivileged, AppArmor, seccomp)
- You want **real storage** (ZFS, Btrfs, Ceph, Linstor DRBD copy-on-write snapshots)
- You want a clean, unified platform for **OCI microservices, system containers, and VMs**



Project Status & Community

- **Destination:** Official Linux Containers project (`lxc/incus-compose`)
- **Current Version:** `v1.3.x` (active, rapid development)
- **Installation:**

```
# Standalone installer  
curl -sSfL https://raw.githubusercontent.com/lxc/incus-compose/main/install.sh | sh -s -- -b ~/.local/bin  
  
# Arch Linux AUR: incus-compose-bin  
# Debian/Ubuntu: incus-extra package via zabbly repository
```

- **Documentation:** incus-compose.org
- **GitHub:** github.com/lxc/incus-compose
- **Forum:** discuss.linuxcontainers.org



Questions & Discussion

Thank you!

- **GitHub:** `github.com/lxc/incus-compose`
- **Docs:** `incus-compose.org`
- **Community Forum:** `discuss.linuxcontainers.org`